Программная инженерия

Лекция 7: Основы оркестрации, docker-compose

Денисов Артём Владимирович

Уральский Федеральный Университет

2026

Рекап

Что мы уже прошли?

- Контейнеризация и её назначение
- Базовая работа с Docker
- Dockerfile и сборка образов
- Docker Compose для нескольких сервисов

Базовые сущности Docker

Из чего состоит Docker

- Image — шаблон/образ приложения
- Container — запущенный экземпляр. Реализация шаблона

Dockerfile как описание окружения

Dockerfile имеет декларативный вид

- Каждая инструкция описывает желаемое состояние слоя образа
- Окружение описывается как код
- Сборка становится воспроизводимой

Ключевые инструкции

- `FROM`, `WORKDIR`, `COPY`
- `RUN`, `CMD`
- `ENV`, `EXPOSE`

Docker Layers

Определение

- Каждая инструкция Dockerfile создаёт отдельный слой

Ключевые характеристики

- Слои кешируются
- Повторная сборка ускоряется
- Порядок инструкций влияет на эффективность

Multi-stage build

Что это?

- Разделение сборки и запуска
- Несколько этапов в одном Dockerfile
- Финальный образ содержит только нужное

Зачем это нужно?

- Уменьшить размер образа
- Убрать лишние инструменты из production сборки

Контейнеризация и виртуализация

Виртуальная машина

- Отдельная гостевая ОС
- Больше накладных расходов
- Медленнее старт

Контейнер

- Изолированный процесс от общего ядра хоста
- Быстрее и легче

Почему на Windows и macOS нужна VM для Docker

Docker опирается на возможности Linux ядра

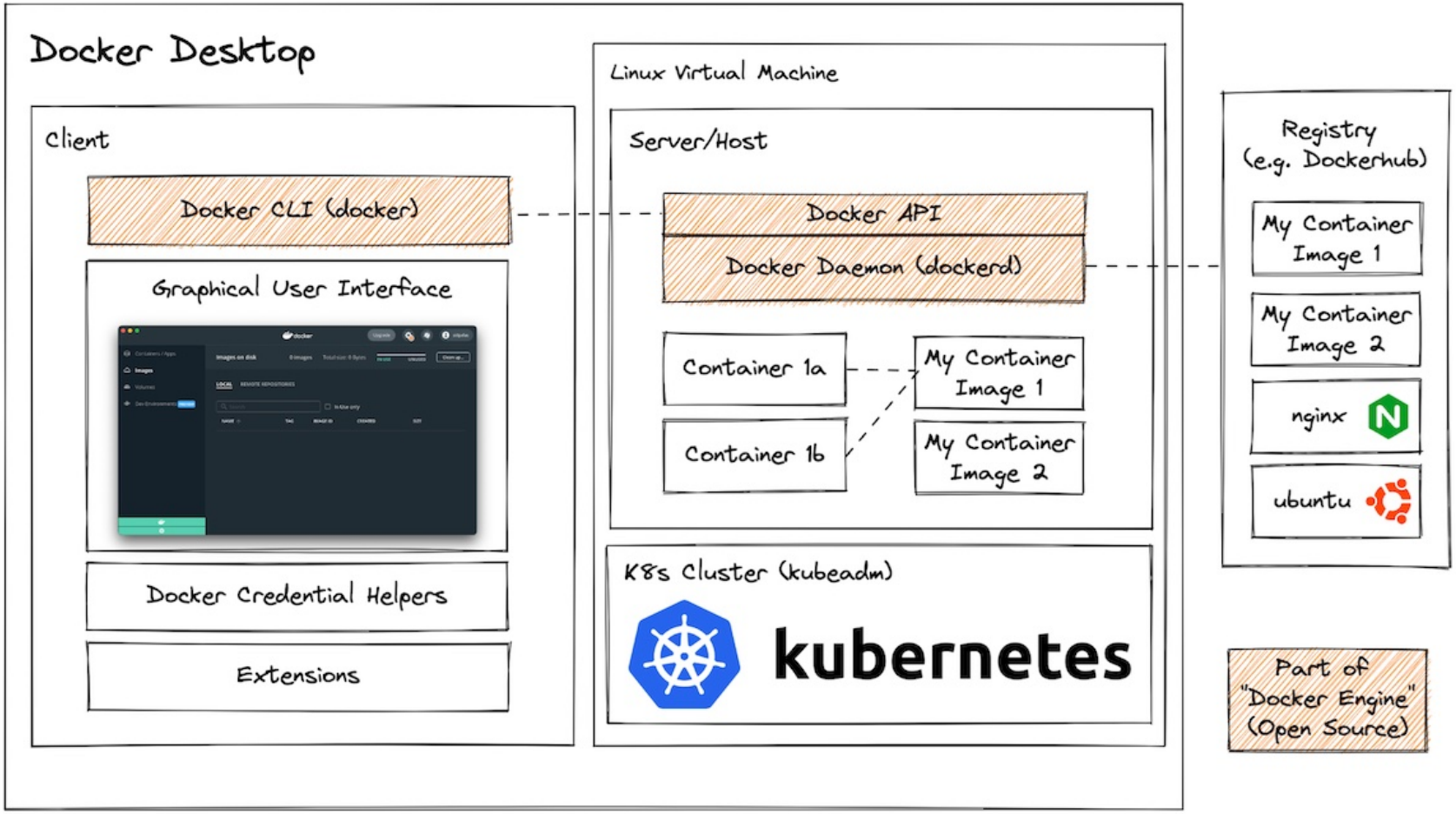
- Контейнеры используют механизмы ядра ОС
- Без Linux-ядра нельзя запускать Linux-контейнеры напрямую

Что происходит на Windows и macOS

Особенность других ОС

- Windows и macOS нет Linux-ядра
- Сначала поднимается небольшая Linux VM
- Контейнеры работают внутри VM

Docker Desktop архитектура



Что это значит на практике

Команды остаются теми же

- `docker build`
- `docker run`
- `docker compose up`

Практический вывод

- на macOS и Windows есть слой виртуализации
- Docker Desktop инкапсулирует эту сложность
- Docker Desktop использует Linux через WSL (Windows) или VM (macOS)
- Поведение остаётся единым на разных ОС

Образы Docker не всегда кроссплатформенные

Главная мысль

- Docker-образ зависит не только от приложения
- Важна целевая платформа сборки
- `amd64` и `arm64` — разные среды

Почему так происходит?

- Процессоры исполняют разный машинный код
- Один образ нельзя считать универсальным по умолчанию

Один исходный код — разные бинарники

```
1  #include <iostream>
2  int main() { std::cout << "Hello"; }
3
```

Сборка под x86

- Бинарник для Intel / AMD
- Инструкции под amd64

Сборка под ARM64

- Бинарник для Apple Silicon
- Инструкции под arm64

Вывод

- Один и тот же исходный код компилируется в разный машинный код.
- Бинарник под amd64 не обязан запускаться на arm64.

Это не проблема Docker

Контейнер

- Не эмулирует железо

По сути: это просто процессы с изоляцией

Виртуальная машина

- Эмулируют или виртуализируют железо
- Сильнее скрывает различия платформ
- Выше уровень изоляции

По сути: это полноценный отдельный компьютер внутри компьютера

Ключевой вывод

Архитектурные различия не исчезают только из-за контейнеризации.

Docker - не магия

Главные тезисы

- Один Dockerfile не гарантирует универсальный образ
- Для `amd64` и `arm64` нужны отдельные сборки
- Для этого используют multi-platform images
- Типичный инструмент — `docker buildx`

Запомнить

“Write once, run anywhere” работает только в пределах совместимой платформы

Контейнеры эфемерны

Что это?

- Контейнер — временный экземпляр приложения
- Контейнер легко пересоздаётся, без следов удаляется

Важно

- Контейнер не рассматривается как отдельная машина. Это *процесс с изоляцией*

Что значит “эфемерный”

Определение

Эфемерность — это принцип проектирования, при котором контейнеры являются временными и легко заменяемыми объектами, не содержащими критических данных и способными быстро запускаться и останавливаться

Ключевые характеристики

- Потеря одного контейнера не критична
- Сбойный контейнер не чинят вручную
- Вместо него запускают новый экземпляр
- Внутренние изменения исчезают после удаления

Практические правила работы с контейнерами

Правило

- Состояние и настройки не должны жить только внутри образа

Хорошо

- Описывать окружение в `Dockerfile`
- Передавать конфигурацию явно

Плохо

- Хранить важные данные внутри контейнера
- Настраивать сервис вручную через `docker exec`

Данные в контейнерах

Где должны жить данные?

- Постоянные данные выносятся за пределы контейнера
- Контейнер должен быть stateless

Типичные варианты

- Volumes
- Базы данных
- Внешние файловые хранилища

Эфемерность

Главные тезисы

- Контейнер — воспроизводимая единица запуска
- Контейнер должен быть легко заменяемым
- Важное состояние хранится вне контейнера

Docker Compose

Что это?

- Описание многоконтейнерного приложения
- Единый YAML-файл для сервисов
- Вся информация о сервисе (сети, тома и переменные) в одном месте

Зачем это нужно?

- Запускать систему одной командой
- Воспроизводить локальное окружение

Где заканчивается Docker Compose

Compose хорошо подходит

- Локальная разработка
- Тестовые стенды
- Небольшие проекты

Production требует

- Несколько хостов
- Самовосстановление
- Масштабирование
- Зашифрованные секреты
- Обновления без простоя

Какие проблемы возникают в реальной системе

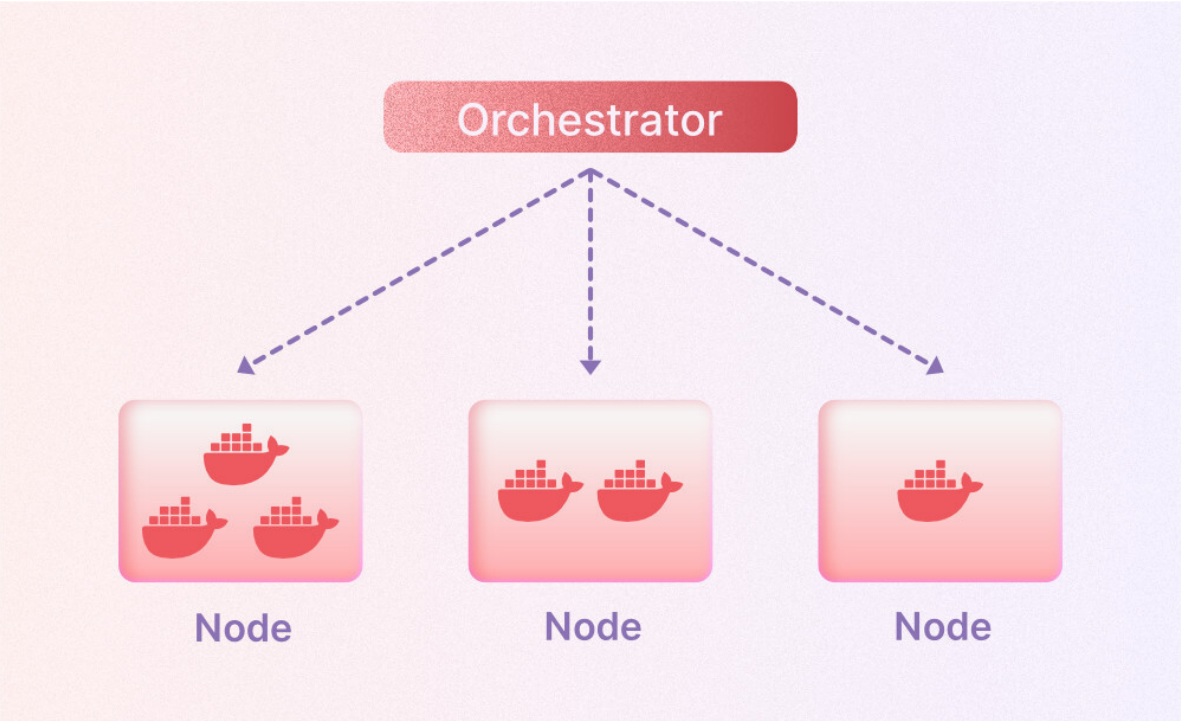
Типовые эксплуатационные сбои

- Контейнер может упасть
- Одного экземпляра мало под нагрузкой
- Сервер может выйти из строя
- Сервисы должны находить друг друга

Вывод

Ручное управление контейнерами плохо масштабируется.

Что такое оркестрация контейнеров



Оркестрация — это

- Автоматическое управление множеством контейнеров

Главная идея: desired state

Мы описываем цель

- backend должен иметь 3 реплики
- Сервис должен быть доступен по сети
- Сбойный экземпляр нужно восстановить

Роль оркестратора

Оркестратор поддерживает фактическое состояние близким к желаемому

Какие задачи решает оркестратор

Основные функции

- Размещение контейнеров по узлам
- Поддержание числа реплик
- Самовосстановление
- Масштабирование сервисов
- Service discovery
- Обновления и откаты

Размещение контейнеров по узлам

Scheduling

- Кластер состоит из нескольких машин
- Оркестратор решает, где запускать контейнер
- Учитываются CPU, RAM и состояние узлов

Самовосстановление и реплики

Идея

- Сервис работает в нескольких копиях
- Каждая копия — это реплика
- Одна реплика дала сбой - работает другая

Что делает оркестратор

- Обнаруживает неисправный контейнер
- Поднимает новую реплику
- Сохраняет нужное число экземпляров

Масштабирование сервиса

Под нагрузкой

- Одного экземпляра может не хватить
- Нужно увеличить число реплик

Оркестратор позволяет

- Увеличивать число экземпляров при большой нагрузке
- Уменьшать число экземпляров при малой нагрузке
- Делать это вручную или автоматически

Service discovery и стабильная точка доступа

Проблема

- Контейнеры эфемерны
- Их IP-адреса меняются
- Адреса нельзя жёстко зашивать в код

Решение

- Сервис получает стабильное имя
- Трафик распределяется между репликами

Обновления без простоя

Что нужно в production

- Нельзя просто выключить старую версию
- Обновление должно быть управляемым

Что делает оркестратор

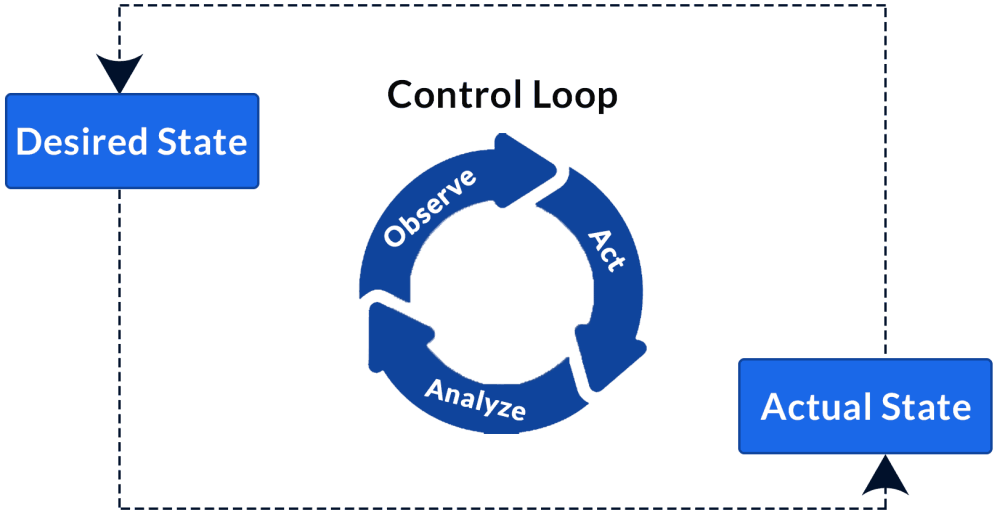
- Запускает новые экземпляры постепенно
- Удаляет старые после проверки
- Позволяет выполнить rollback

Базовые сущности оркестрации

Минимальная модель

- **Кластер** — группа машин
- **Узел** — отдельная машина
- **Workload** — то, что запускаем
- **Реплика** — одна копия приложения
- **Сервис** — стабильная точка доступа

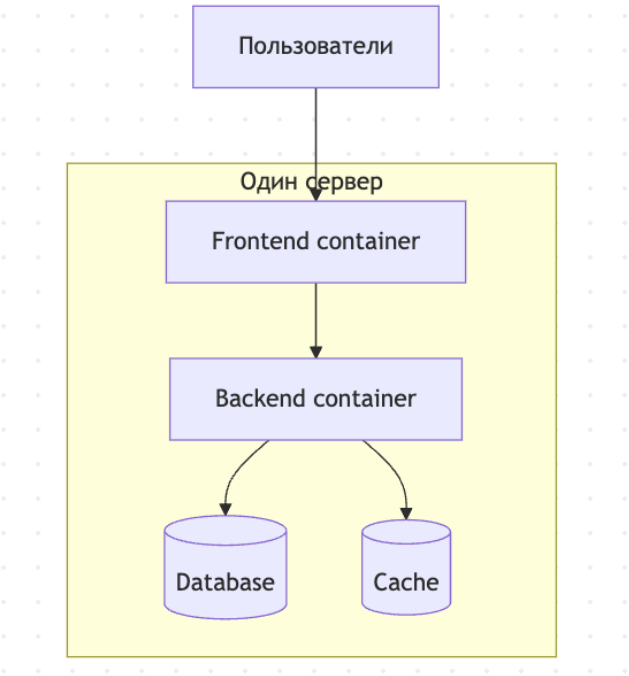
Control loop: как мыслит оркестратор



Цикл: “проверил → сравнил → починил → повторил”

- Пользователь задаёт desired state
- Система сравнивает: что есть и что должно быть
- Расхождения автоматически исправляются

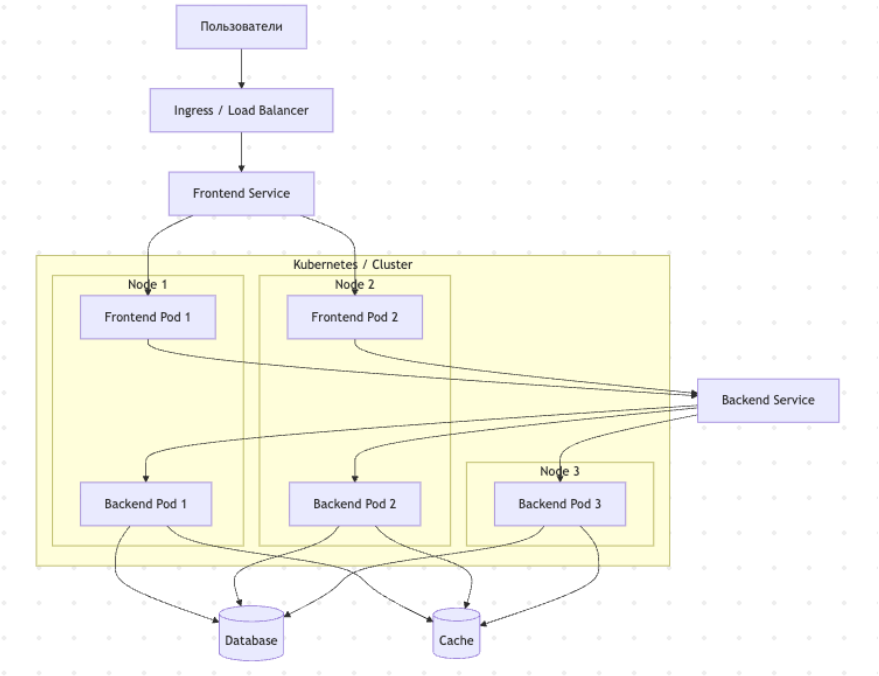
Пример: интернет-магазин без оркестрации



Проблемы

- Backend падает -> сервис недоступен
- Всплеск трафика -> перегрузка
- Обновление ломает систему

Пример: интернет-магазин с оркестрацией



Решение

- Несколько реплик backend -> отказоустойчивость
- Автоматическая замена сбойных экземпляров
- Постепенное выкатывание новой версии

Оркестраторы

Какие оркестраторы существуют

- Docker Swarm - встроен в Docker
- Kubernetes - мощный, популярный
- Nomad
- Amazon ECS
- ...

Итог темы

Главные тезисы

- Compose помогает запускать несколько контейнеров
- Оркестрация управляет системой контейнеров в production
- Оркестратор поддерживает desired state
- Он решает масштабирование, сбои, сеть и обновления

Безопасность контейнеров

Image security

- Что находится внутри образа
- Уязвимости и лишние зависимости
- Секреты, теги, подпись

Runtime security

- Что происходит после запуска
- Ограничение привилегий
- Защита хоста и среды выполнения

Image security: уменьшение поверхности атаки

Главная идея

- Образ должен быть как можно меньше
- Внутри не должно быть лишнего

Практики

- Использовать минимальные base images
- Применять multi-stage builds
- Не ставить лишние пакеты
- Не включать dev dependencies в production

Image security: доверие и воспроизводимость

Что важно контролировать

- Сканировать образы на уязвимости
- Запускать процессы от непривилегированного пользователя
- Не хранить секреты внутри образа

Целостность образа

- Подписывать () образы
- Фиксировать версии через `major.minor`

Runtime security: ограничение последствий взлома

Главный вопрос

- Что сможет сделать атакующий после компрометации?
- Насколько легко ему будет выйти за пределы контейнера?

На уровне Docker daemon

- Усиливать безопасность нужно не только внутри контейнера
- Полезно включать `-userns-remap`
- Это снижает риск для хоста

Runtime security: ограничения для контейнера

Минимум привилегий

- Использовать read-only filesystem, если запись не нужна
- Сначала `-cap-drop=all`
- затем добавлять нужные правила

Дополнительные ограничения

- Ограничивать CPU и память
- Применять `-security-opt`

Источники информации и изображений

1. https://middleware.io/blog/what-is-container-orchestration/
2. https://k21academy.com/kubernetes/kubernetes-operator/
3. https://github.com/sidpalas/devops-directive-docker-course

Программная инженерия

Лекция 7: Основы оркестрации, docker-compose

Денисов Артём Владимирович

Уральский Федеральный Университет

2026